

Rod Cutting Problem

Given a rod of length n and the **prices** of its pieces of different **lengths** from 1 to n . Design a recursive algorithm to find the **maximum profit** you can make by cutting the rod and selling its pieces.

Example

Input: $n=8$

length	1	2	3	4	5	6	7	8
prices	1	5	8	9	10	17	17	20

Output: 22

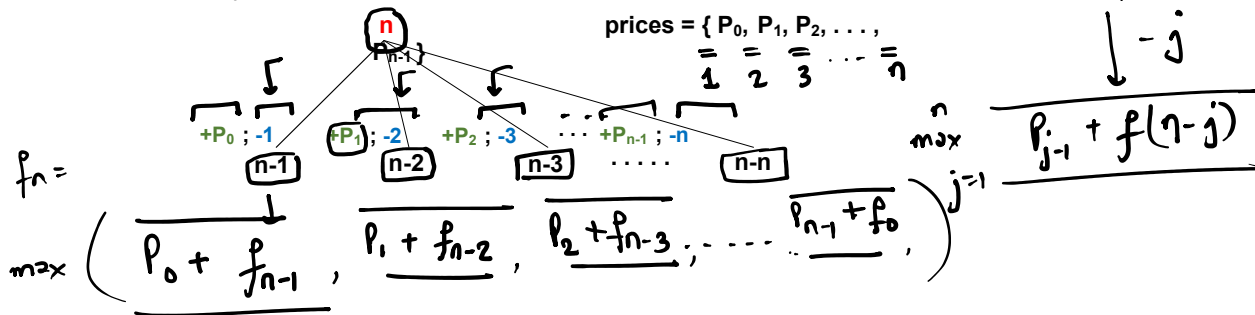
Explanation: $5(2) + 17(6) = 22$

$$\begin{array}{c} 3 \quad 5 \\ \downarrow \quad \downarrow \\ \text{€}8 \quad \text{€}10 = \text{€}18 \end{array}$$

$$\begin{array}{c} 2 \quad 3 \quad 1 \quad 2 \\ \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \\ \text{€}5 \quad \text{€}8 \quad \text{€}1 \quad \text{€}5 = \text{€}19 \end{array}$$

$$\begin{array}{c} 2 \quad 6 \\ \downarrow \quad \downarrow \\ \text{€}5 \quad \text{€}17 = \text{€}22 \end{array}$$

$f_n \rightarrow$ it is a fn that finds max profit we can make from a rod of length n



Recursive Case

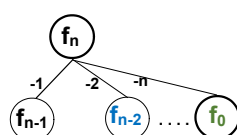
Let $f(n)$ be a function which computes the maximum profit one can make by cutting a rod of length n and selling its pieces then

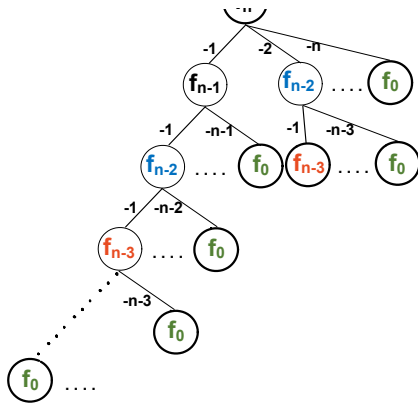
$$f(n) = \max_{j=1}^n (P[j-1] + f(n-j))$$

Base Case

$$f(0) = 0$$

What is the reason behind **exponential** time complexity?

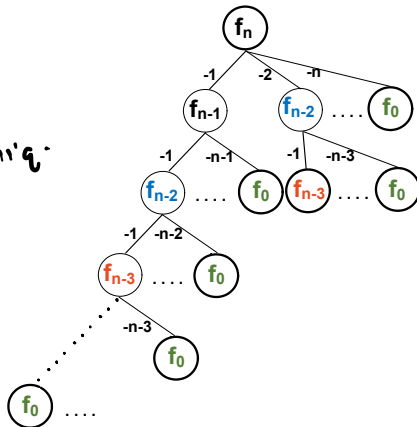




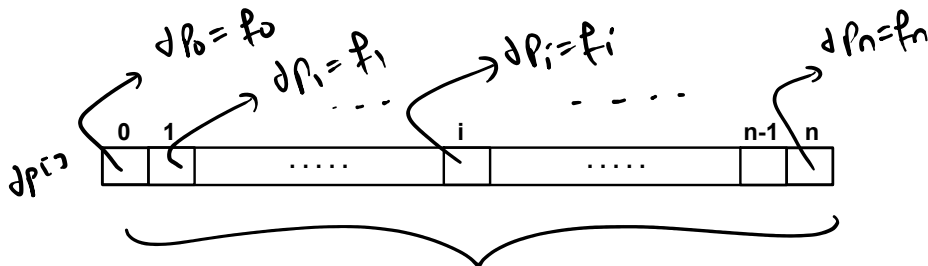
Top-Down Dynamic Programming

f_0 to f_n

$n+1$ unique



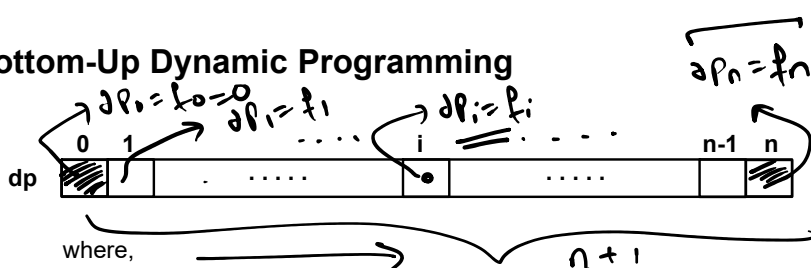
init: -1



where,

$f(i)$ is a function which computes the maximum profit one can make by cutting a rod of length i and selling its pieces.

Bottom-Up Dynamic Programming



where,

$f(i)$ is a function which computes the maximum profit one can make by cutting a rod of length i and selling its pieces.

From recurrence,

$$f(n) = \max_{1 \leq i \leq n} \{f(n-i) + p_i\}$$

$dp_i \rightarrow f_i$
 ↓
 find me max profit for 0 rod of len- i

can make by cutting a rod of length i and selling its pieces.

From recurrence,

$$f(i) = \max_{j=1}^i p[j-1] + f(i-j) \quad ; i > 0$$

Therefore,

$$\begin{aligned} dp[i] &= \max_{j=1}^i p[j-1] + dp[i-j] \\ &= \end{aligned}$$

$$\max_{k=1}^i p[k-1] + f[i-k]$$

$$f_n = \max_{j=1}^n p_{j-1} + f_{n-j}$$

$$\Rightarrow f_i = \max_{j=1}^i p_{j-1} + f_{i-j}$$

Partition Array for Maximum Sum

Given an integer array, **partition the array** into (contiguous) subarrays of length at most **K** such that after the partitioning each subarray has their value changed to the **maximum value** of the subarray. Design an algorithm to find the **largest sum** of the given array after partitioning.

Input : K = 3

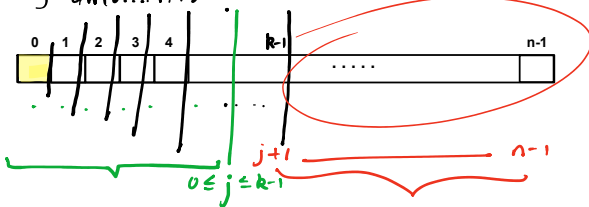
0	1	2	3	4	5	6
1	15	7	9	2	5	10

o/p: 84

$$\begin{array}{c}
 \text{K=3} \\
 \begin{array}{|c|c|c|}
 \hline
 1 & 15 & 7 \quad 9 \quad 2 \\
 \hline
 \end{array}
 \begin{array}{|c|c|}
 \hline
 5 & 10 \\
 \hline
 \end{array} \\
 \begin{array}{|c|c|c|}
 \hline
 2 \leq 3 & 3 \leq 3 & 2 \leq 3 \\
 \hline
 \end{array} \\
 \begin{array}{|c|c|c|}
 \hline
 15 & 15 & 9 \quad 9 \quad 9 \\
 \hline
 \end{array}
 \begin{array}{|c|c|}
 \hline
 10 & 10 \\
 \hline
 \end{array} \\
 \hline
 77 \quad \checkmark
 \end{array}$$

$$\begin{array}{c}
 \begin{array}{|c|c|c|c|}
 \hline
 1 & 15 & 7 & 9 \\
 \hline
 \end{array}
 \begin{array}{|c|}
 \hline
 2 \\
 \hline
 \end{array}
 \begin{array}{|c|c|c|}
 \hline
 5 & 10 & 10 \\
 \hline
 \end{array} \\
 \begin{array}{|c|c|c|c|}
 \hline
 3 \leq 3 & 1 \leq 3 & 3 \leq 3 & \\
 \hline
 \end{array} \\
 \begin{array}{|c|c|c|c|}
 \hline
 15 & 15 & 15 & 9 \\
 \hline
 \end{array}
 \begin{array}{|c|c|c|}
 \hline
 10 & 10 & 10 \\
 \hline
 \end{array} \\
 \hline
 84
 \end{array}$$

find the max. sum from partitioning arr[0... n-1] ?



$$\begin{array}{c}
 \begin{array}{|c|c|c|c|c|c|}
 \hline
 0 & \dots & i-1 & i & i+1 & \dots & n-1 \\
 \hline
 \end{array} \\
 \begin{array}{l}
 \underbrace{(\max_{i \dots j} * (j-i+1))}_{\text{max sum from partitioning arr[i... n-1]}} \\
 + \\
 f(j+1)
 \end{array} \\
 f(i) = \text{find the max. sum from partitioning arr[i... n-1]} \\
 \text{to solve this decide the next cut}
 \end{array}$$

$$\begin{array}{c}
 \begin{array}{|c|c|c|c|}
 \hline
 i & j & j+1 & \dots & n-1 \\
 \hline
 \end{array} \\
 \begin{array}{l}
 \max_{i \dots j} * (j-i+1) \\
 + \\
 f(j+1)
 \end{array} \\
 f(i) = \max_{j=i}^{n-1} (\max_{i \dots j} * (j-i+1) + f(j+1))
 \end{array}$$

Recursive Case

Let $f(i)$ be a function which finds the **largest sum** of the given array that starts at the i^{th} index after partitioning then,

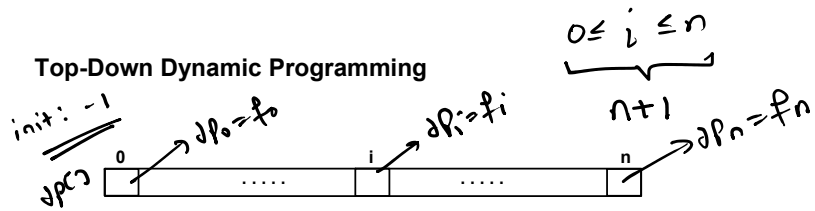
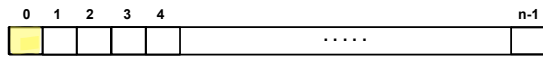
$$f(i) = \max_{j=i}^{i+K-1} \{ \max_{l=i \dots j} (\text{arr}[l \dots j]) * (j-i+1) + f(j+1) \}$$

Base Case

init $i=0$

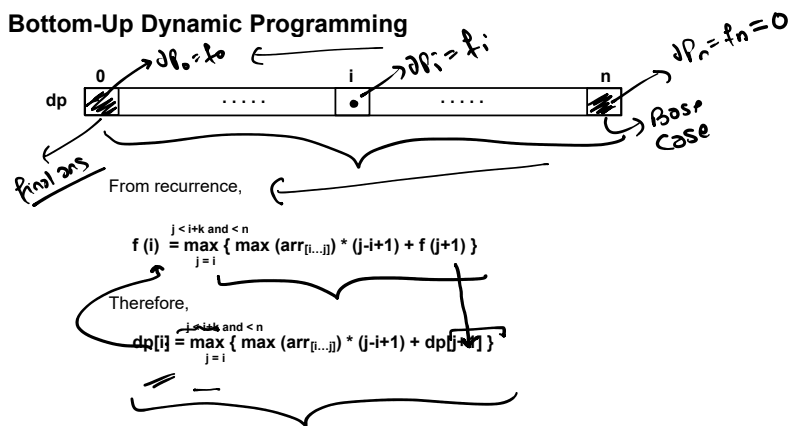
$$f(i=n) = 0$$

What is the reason behind **exponential** time complexity ?



where,

$f(i)$ is a function which finds the **largest sum** of the given array that starts at the i^{th} index after partitioning



[Tenzing and Balls](#)

C. Tenzing and Balls

time limit per test: 1 second

memory limit per test: 256 megabytes

Enjoy solving Tenzing, identified as Accepted!

Tenzing has n balls arranged in a line. The color of the i -th ball from the left is a_i .

Tenzing can do the following operation any number of times:

- select i and j such that $1 \leq i < j \leq |a|$ and $a_i = a_j$.
- remove $a_i, a_{i+1}, \dots, a_j$ from the array (and decrease the indices of all elements to the right of a_j by $j - i + 1$).

Tenzing wants to know the maximum number of balls he can remove.

Input

Each test contains multiple test cases. The first line of input contains a single integer t ($1 \leq t \leq 10^5$) — the number of test cases. The description of test cases follows.

The first line contains a single integer n ($1 \leq n \leq 2 \cdot 10^5$) — the number of balls.

The second line contains n integers $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq n$) — the color of the balls.

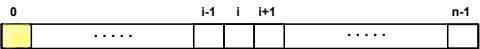
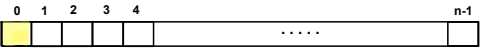
It is guaranteed that sum of n of all test cases will not exceed $2 \cdot 10^5$.

Output

For each test case, output the maximum number of balls Tenzing can remove.

Example

input	Copy
2	
3 2 2 3 3	
4	
2 2 3 2	
output	Copy
4	
3	



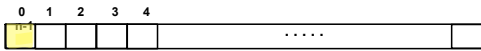
Recursive Case

Let $f(i)$ be a function that find the maximum no. of balls Tenzing can remove from $a[i..n-1]$ then,

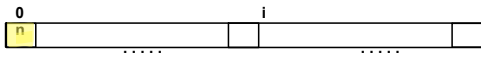
$$f(i) = \max \{ f(i+1), \max_{j=i+1}^{<n} ((j-i+1) + f(j+1)) \}$$
 such that $a[i]$ is equal to $a[j]$

Base Case

What is the reason behind **exponential** time complexity ?



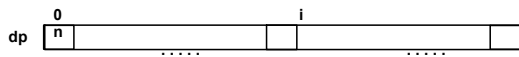
Top-Down Dynamic Programming



where,

$f(i)$ be a function that find the maximum no. of balls Tenzing can remove from $a[i \dots n-1]$ then,

Bottom-Up Dynamic Programming



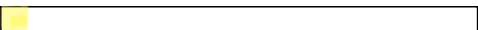
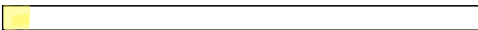
From recurrence,

$$f(i) = \max \{ f(i+1), \max_{j=i+1}^{j < n} (f(j+1) + f(j+1)) \} \text{ such that } a[i] \text{ is equal to } a[j]$$

Therefore,

$$dp[i] = \max \{ dp[i+1], \max_{j=i+1}^{j < n} (dp[j+1] + dp[j+1]) \} \text{ such that } a[i] \text{ is equal to } a[j]$$

Time Optimization Bottom-Up Dynamic Programming



<https://www.youtube.com/watch?v=bXg1-S3hWNY&t=2953s>

Best Time to Buy and Sell Stock IV

You are given an integer array **prices** where **prices [i]** is the price of a given stock on the **ith day**, and an integer **K**. Design an algorithm to find the **maximum profit** you can achieve by completing at most **K** transactions.

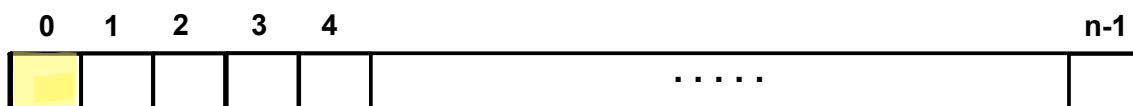
Note : You may not engage in multiple transactions simultaneously i.e. you must sell the stock before you buy again.

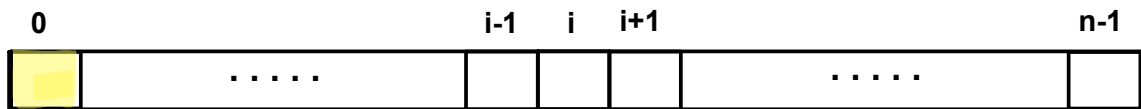
Input : K= 2

	0	1	2
prices	2	4	1

Input : K= 2

	0	1	2	3	4	5
prices	3	2	6	5	0	3





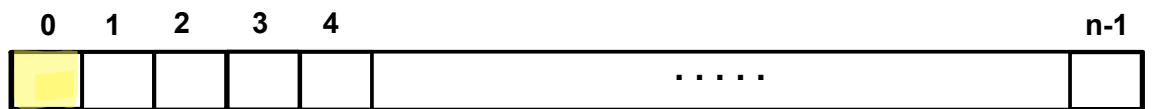
Recursive Case

Let $f(i, k)$ be a function which computes the **maximum profit** one can make by completing at most k transactions starting from the i^{th} then,

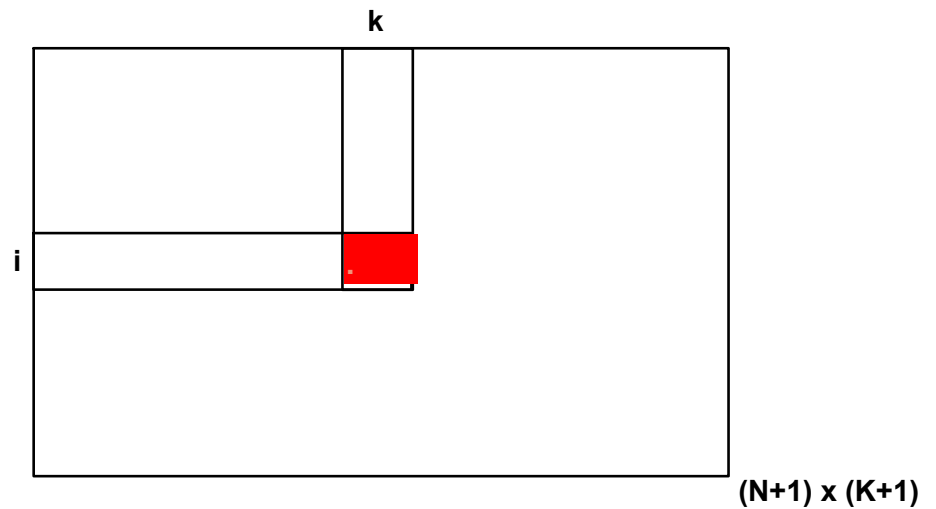
$$f(i, k) = \max \{ f(i+1, k), \max_{j=i+1}^{n-1} \{ \text{prices}[j] - \text{prices}[i] + f(j+1, k-1) \} \}$$

Base Case

What is the reason behind **exponential** time complexity ?



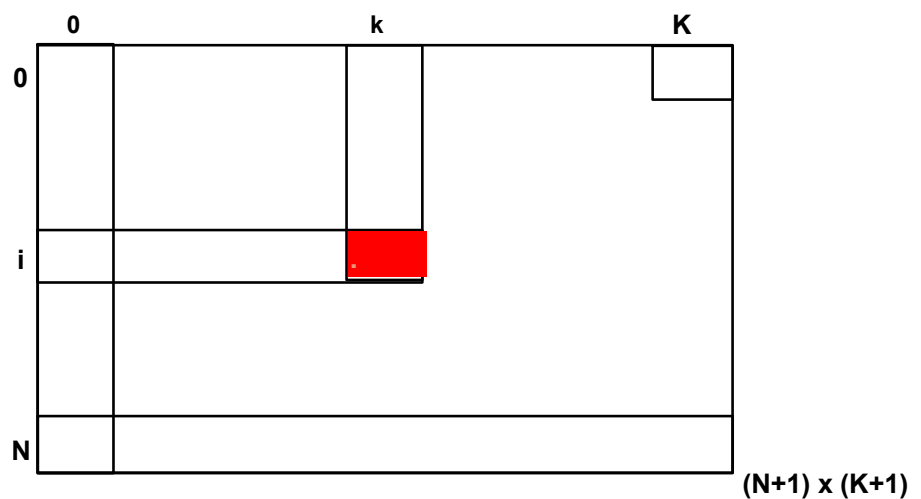
Top-Down Dynamic Programming



where,

$f(i, k)$ is a function which computes the **maximum profit** one can make by completing at most k transactions starting from the i^{th} day

Bottom-Up Dynamic Programming



where,

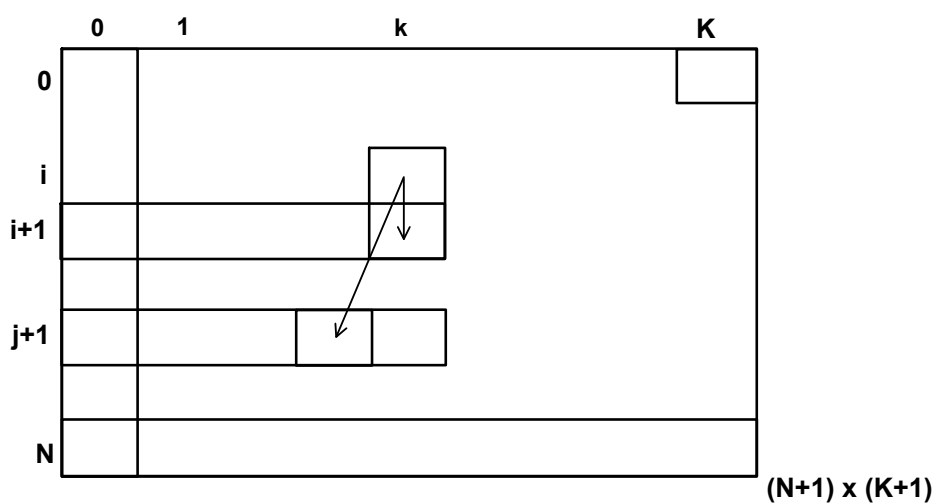
$f(i, k)$ is a function which computes the **maximum profit** one can make by completing at most k transactions starting from the i^{th} day

From recurrence,

$$f(i, k) = \max \{ f(i+1, k), \max_{j=i+1}^{n-1} \{ \text{prices}[j] - \text{prices}[i] + f(j+1, k-1) \} \}$$

Therefore,

$$\text{dp}[i][k] = \max \{ \text{dp}[i+1][k], \max_{j=i+1}^{n-1} \{ \text{prices}[j] - \text{prices}[i] + \text{dp}[j+1][k-1] \} \}$$



<https://www.youtube.com/watch?v=VPlphqIYz1A&t=110s>

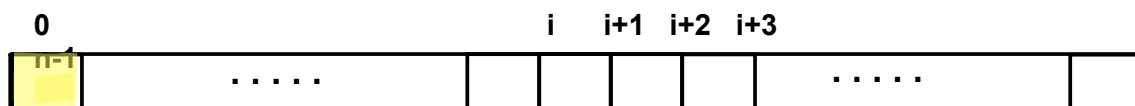
Count Palindromic Partitioning

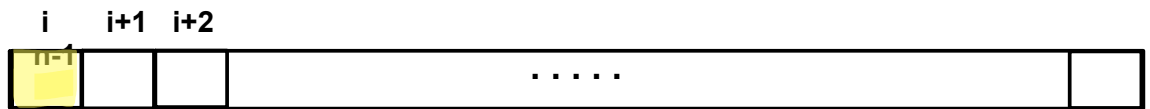
Given a string of length **N**, design a **recursive** algorithm to **count** the number of its **palindromic partitioning**.

Example

Input : **N** = 5

0	1	2	3	4
a	b	a	a	a





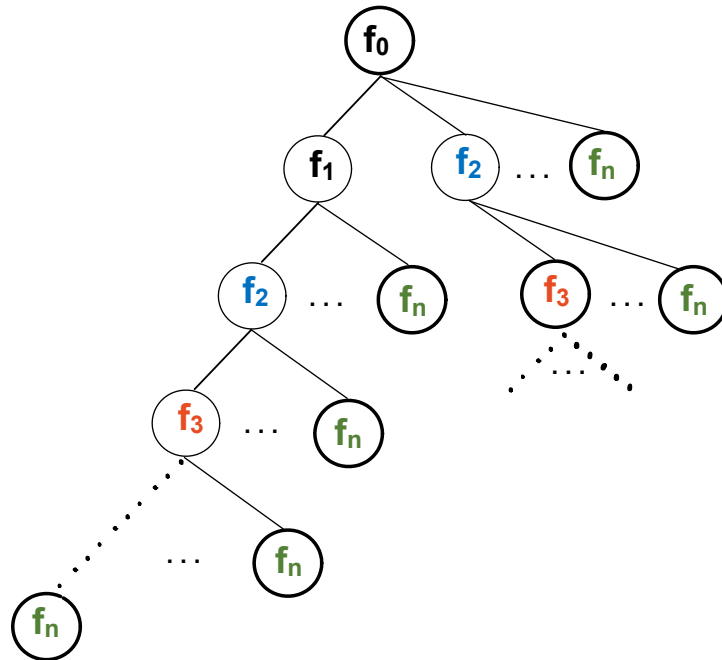
Recursive Case

Let $f(i)$ be a function which **counts** the number of **palindromic** partitioning of $str[i \dots n-1]$ then,

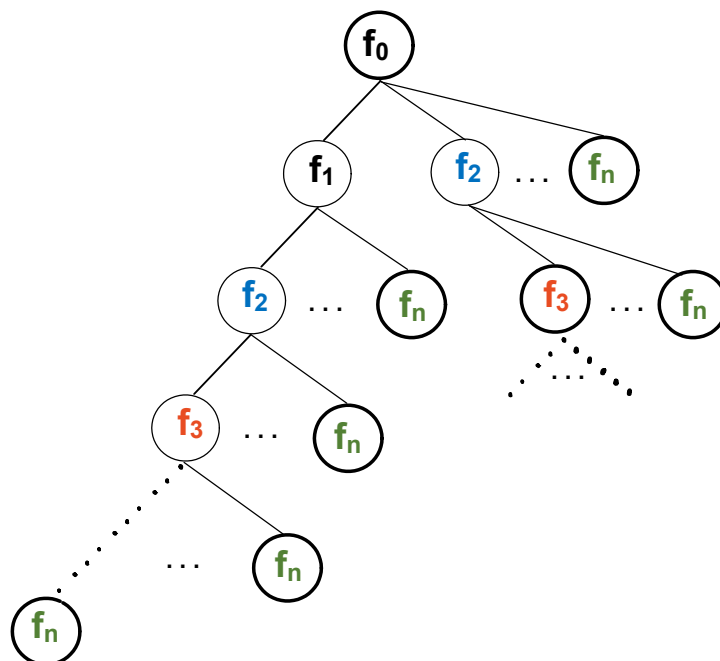
$$f(i) = \sum_{j=i}^{n-1} f(j+1) \text{ such that } str[i \dots j] \text{ is a } \textbf{palindrome}$$

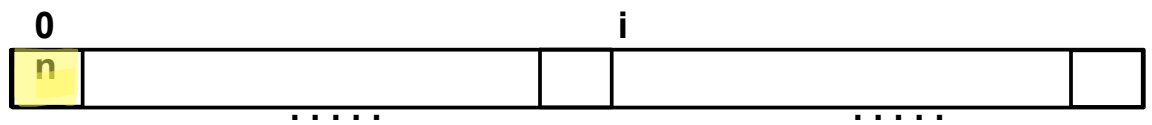
Base Case

What is the reason behind **exponential** time complexity ?



Top-Down Dynamic Programming

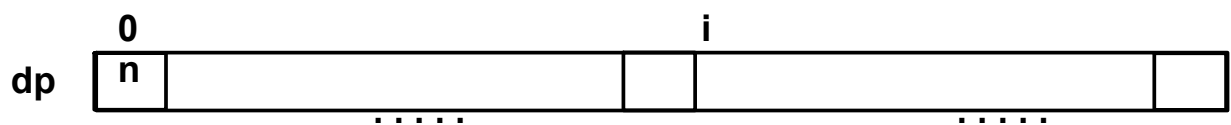




where,

f(i) is a function which **counts** the number of palindromic partitioning of **str[i . . . n-1]**.

Bottom-Up Dynamic Programming



where,

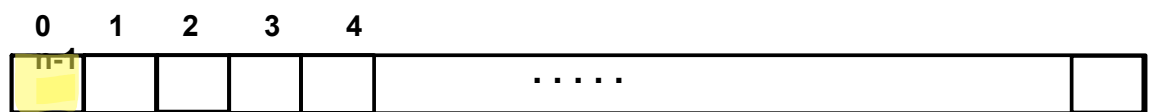
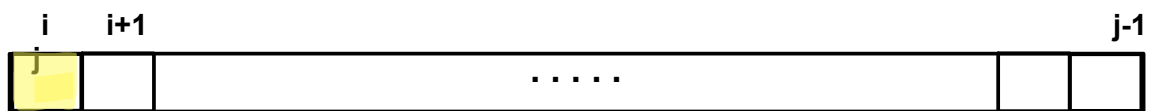
f(i) is a function which **counts** the number of palindromic partitioning of **str[i . . . n-1]**

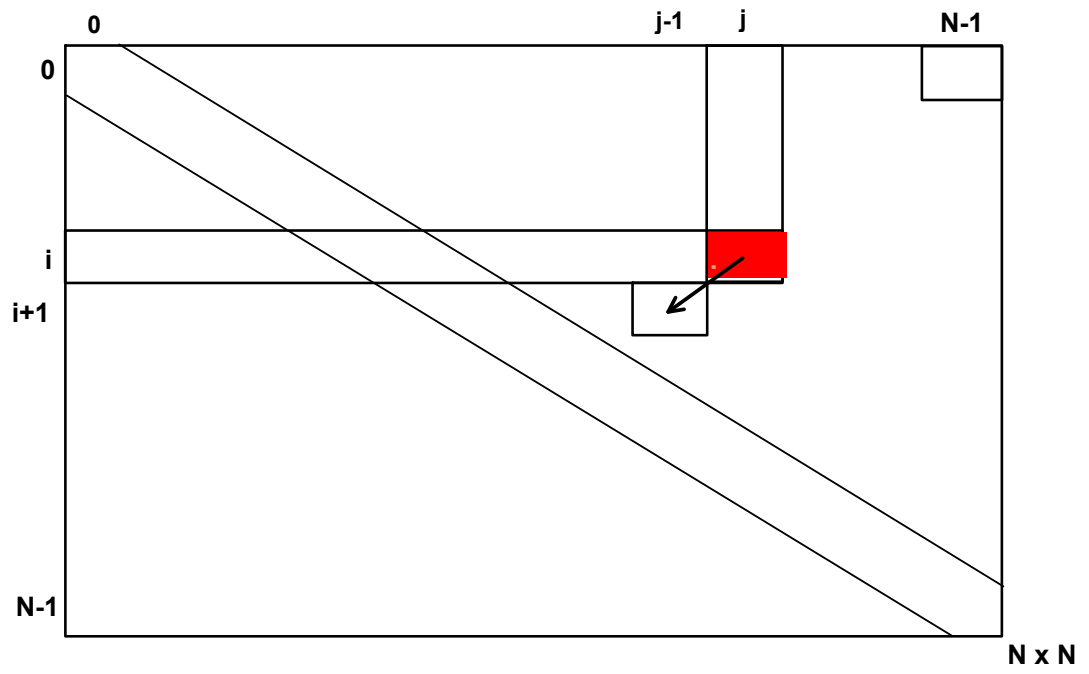
From recurrence,

$$f(i) = \sum_{j=i}^{n-1} f(j+1) \quad \text{such that } \mathbf{str[i \dots j]} \text{ is a palindrome}$$

Therefore,

$$\mathbf{dp[i] = \sum_{j=i}^{n-1} dp[j+1]} \quad \text{such that } \mathbf{str[i \dots j]} \text{ is a palindrome}$$





Try to also solve this problem using binary search

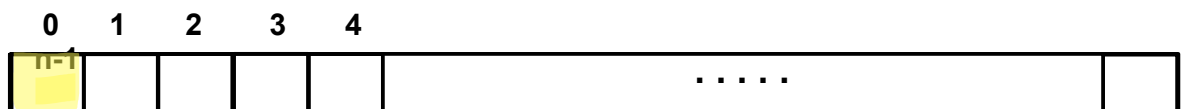
[DP solution] <https://www.youtube.com/watch?v=S6Kp7juvkFA&t=133s>

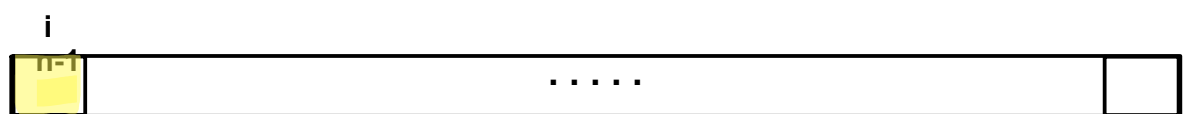
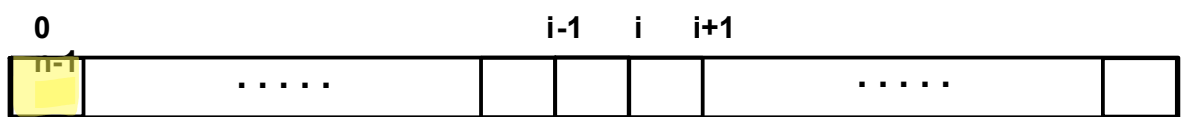
Split Array Largest Sum

Given an integer array **nums**, and an integer **K**, **split the array** into K non-empty subarrays such that the **largest sum** of the any subarray is minimized. Design an algorithm to find the minimized largest sum of the split.

Input : K = 2

0	1	2	3	4
7	2	5	10	8





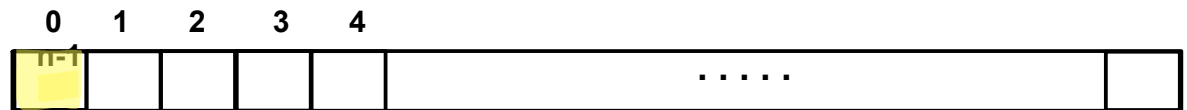
Recursive Case

Let $f(i, k)$ be a function which finds the **minimized largest sum** upon partitioning / splitting $arr[i \dots n-1]$ into k **non-empty** subarrays then,

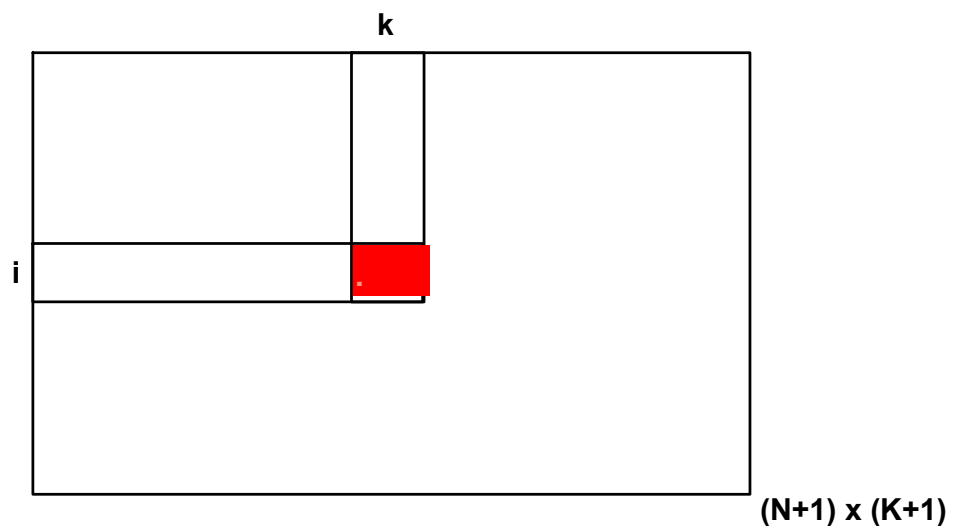
$$f(i, k) = \min_{j=i}^{< n} \{ \max [S_{arr[i \dots j]}, f(j+1, k-1)] \}$$

Base Case

What is the reason behind **exponential** time complexity ?



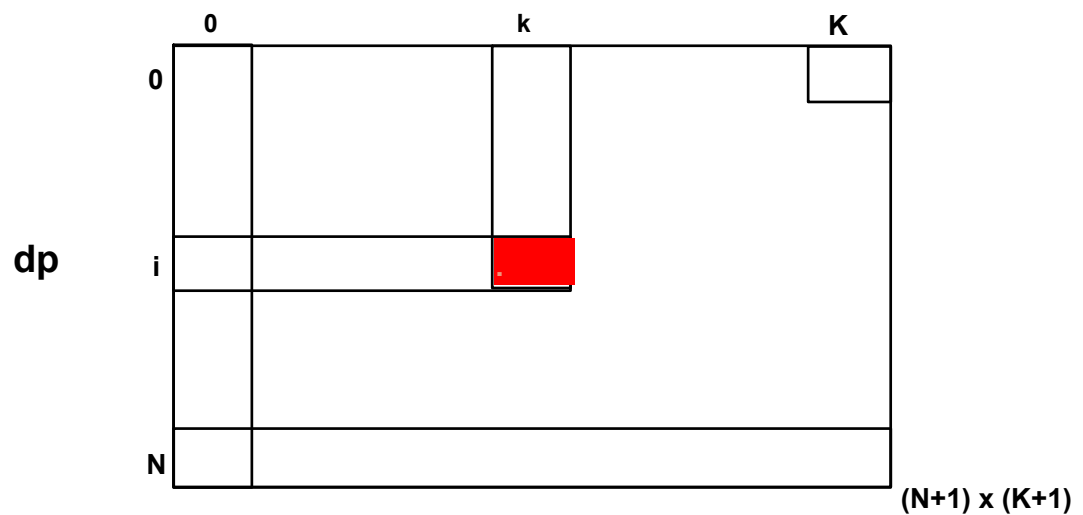
Top-Down Dynamic Programming



where,

$f(i,k)$ is a function which finds the **minimized largest sum** upon partitioning / splitting **$arr[i \dots n-1]$** into **k non-empty** subarrays

Bottom-Up Dynamic Programming



where,

$f(i,k)$ is a function which finds the **minimized largest sum** upon partitioning / splitting $arr[i \dots n-1]$ into **k non-empty** subarrays

From recurrence,

$$f(i, k) = \min_{j=i}^{\leq n-k} \{ \max [S_{arr[i \dots j]}, f(j+1, k-1)] \}$$

Therefore,

$$dp[i][k] = \min_{j=i}^{\leq n-k} \{ \max (S_{arr[i \dots j]}, dp[j+1][k-1]) \}$$

Matrix Chain Multiplication (MCM)

Given a sequence of **N** matrices **{A₁ , A₂ , . . . , A_N}**, design an algorithm to find the **minimum** no. of operations (i.e. scalar multiplications) required to **multiply** them, assume the dimensions of the matrices are compatible for multiplication.

Example

Input : N = 3

0	1	2	3
2	1	2	4

Output : 16

a ₁₁	a ₁₂
a ₂₁	a ₂₂
a ₃₁	a ₃₂

3x2

b ₁₁	b ₁₂	b ₁₃
b ₂₁	b ₂₂	b ₂₃

2x3

c ₁₁	c ₁₂	c ₁₃
c ₂₁	c ₂₂	c ₂₃
c ₃₁	c ₃₂	c ₃₃

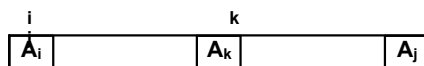
3x3

Input : N = 3

0	1	2	3
2	1	2	4

Input : N = 4

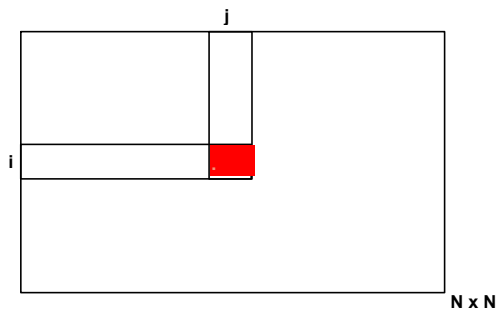
0	1	2	3	4
1	2	1	4	1



Base Case

What is the reason behind **exponential** time complexity ?

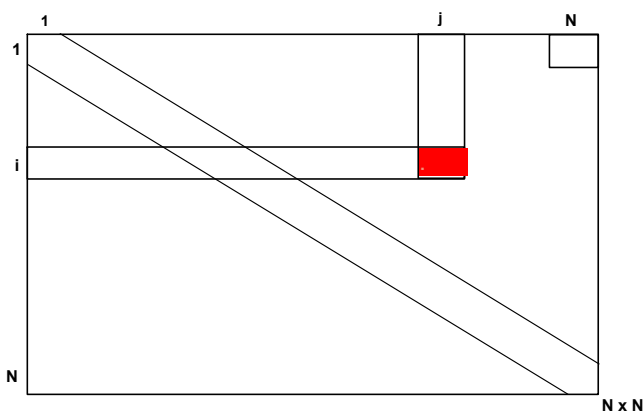
Top-Down Dynamic Programming



where,

$f(i, j)$ is a function which computes the **minimum** no. of operations required to multiply a chain of matrices $\{A_i, A_{i+1}, \dots, A_j\}$

Bottom Up Dynamic Programming



where,

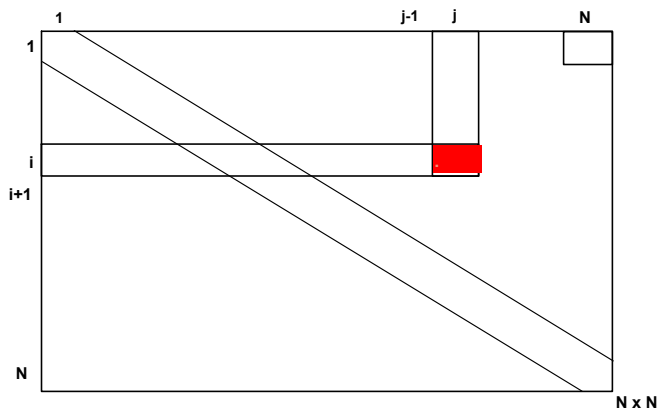
$f(i, j)$ is a function which computes the **minimum** no. of operations required to multiply a chain of matrices $\{A_i, A_{i+1}, \dots, A_j\}$

From recurrence,

$$f(i, j) = \begin{cases} \min_{k=i}^{j-1} \{ f(i, k) + f(k+1, j) + d[i-1] * d[k] * d[j] \} & ; i < j \\ 0 & ; i == j \end{cases}$$

Therefore,

$$dp[i][j] = \begin{cases} \min_{k=i}^{j-1} \{ dp[i][k] + dp[k+1][j] + d[i-1]*d[k]*d[j] \} & ; i < j \\ 0 & ; i == j \end{cases}$$



<https://www.youtube.com/watch?v=gl322KuDul8&t=75s>

Longest Increasing Subsequence (LIS-K) Problem

Given an array of **N** integers and a non-negative integer **K** ($K < N$), design a **recursive** algorithm to compute the **length** of the **longest increasing subsequence** that starts at the **Kth index** of the given integer array.

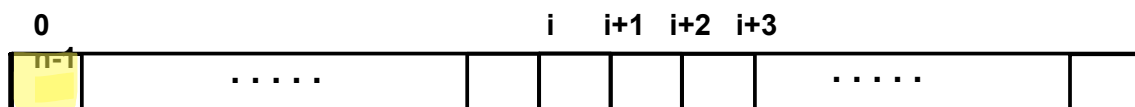
Example

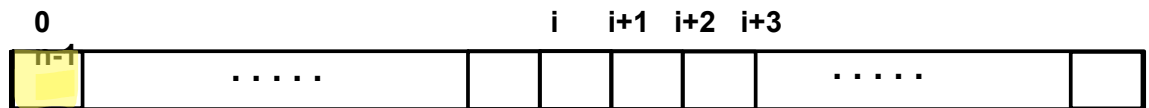
Input : **N** = 7, **K** = 1

0	1	2	3	4	5	6
1	2	5	3	0	4	6

Output : **4**

Explanation : The **LIS** starting at **1st** index is {2, 3, 4, 6}





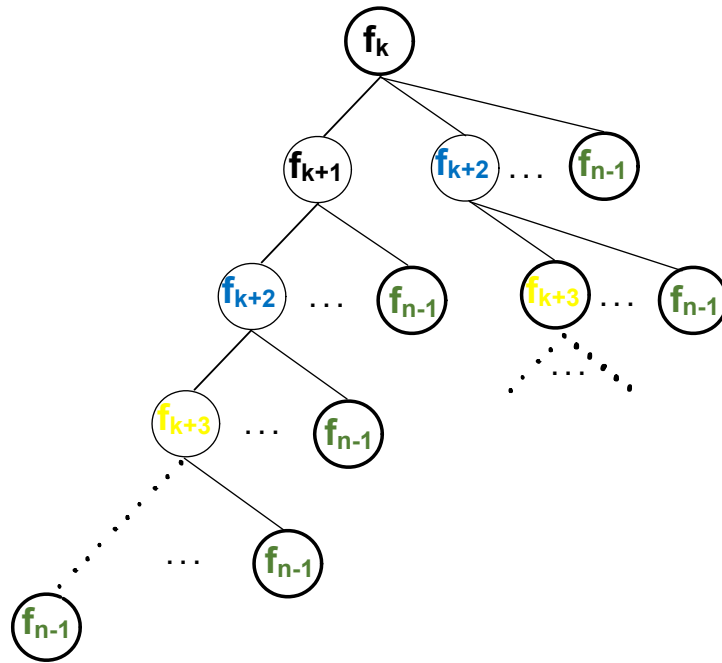
Recursive Case

Let $f(i)$ be a function which computes the **length** of the **longest increasing subsequence** that starts at the i^{th} index of the given array of integers then,

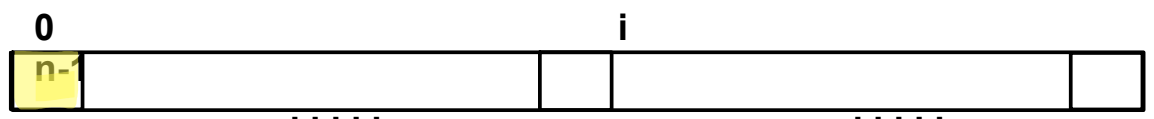
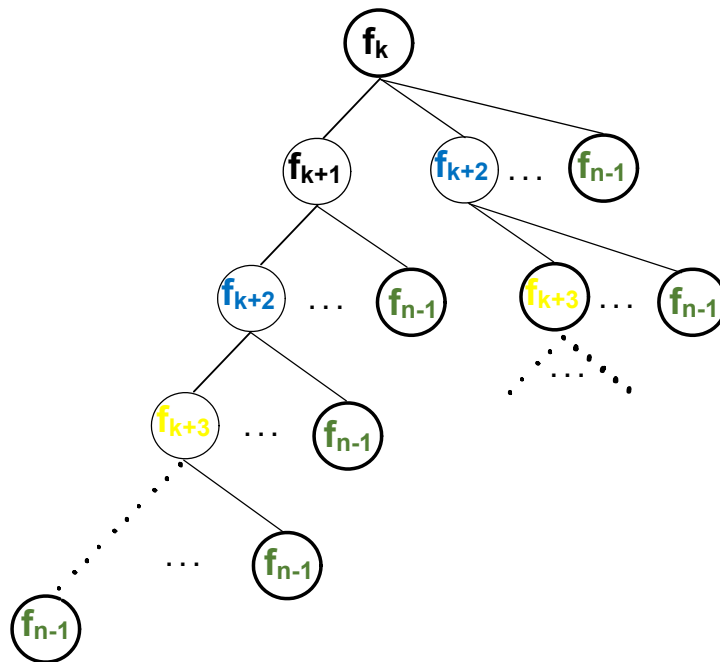
$$f(i) = 1 + \max_{j=i+1}^{n-1} f(j) \quad ; \quad A[j] \geq A[i]$$

Base Case

What is the reason behind **exponential** time complexity ?



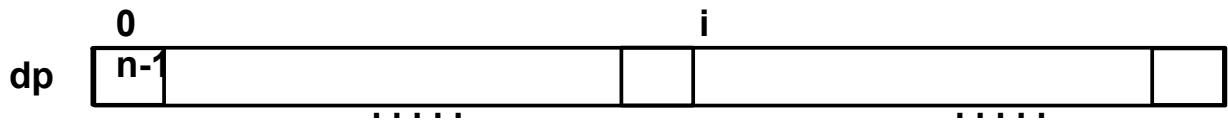
Top-Down Dynamic Programming



where,

f(i) is a function which computes the **length** of the **longest increasing subsequence** that starts at the **ith** index of the given array of integers.

Bottom-Up Dynamic Programming



where,

f(i) is a function which computes the **length** of the **longest increasing subsequence** that starts at the **ith** index of the given array of integers.

From recurrence,

$$f(i) = 1 + \max_{j=i+1}^{n-1} f(j) \quad ; \quad A[j] \geq A[i], i < n-1$$

Therefore,

$$dp[i] = 1 + \max_{j=i+1}^{n-1} dp[j]$$

Try to solve in $O(n \log n)$

[DP solution] <https://www.youtube.com/watch?v=gl322KuDul8&t=75s>

Longest Increasing Subsequence Problem

Given an array of **N** integers design a **recursive** algorithm to compute the **length** of the **longest increasing subsequence** in the given integer array.

Example

Input : **N** = 7

0	1	2	3	4	5	6
1	2	5	3	0	4	6

Output : **5**

Explanation : The **LIS** starting at **1st** index is {1, 2, 4, 4, 6}